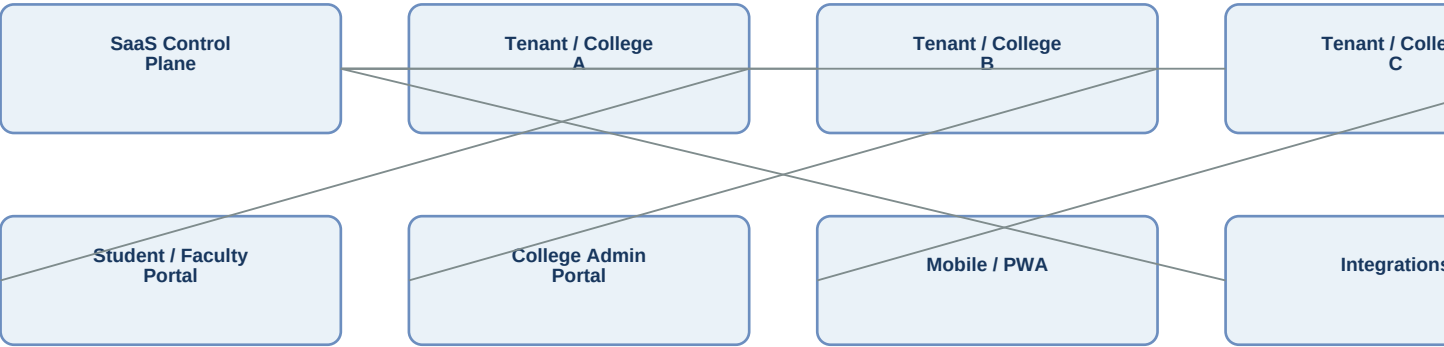


COLLEGE ERP SAAS

Configurable, Multi-Tenant Product Blueprint

One platform → many colleges

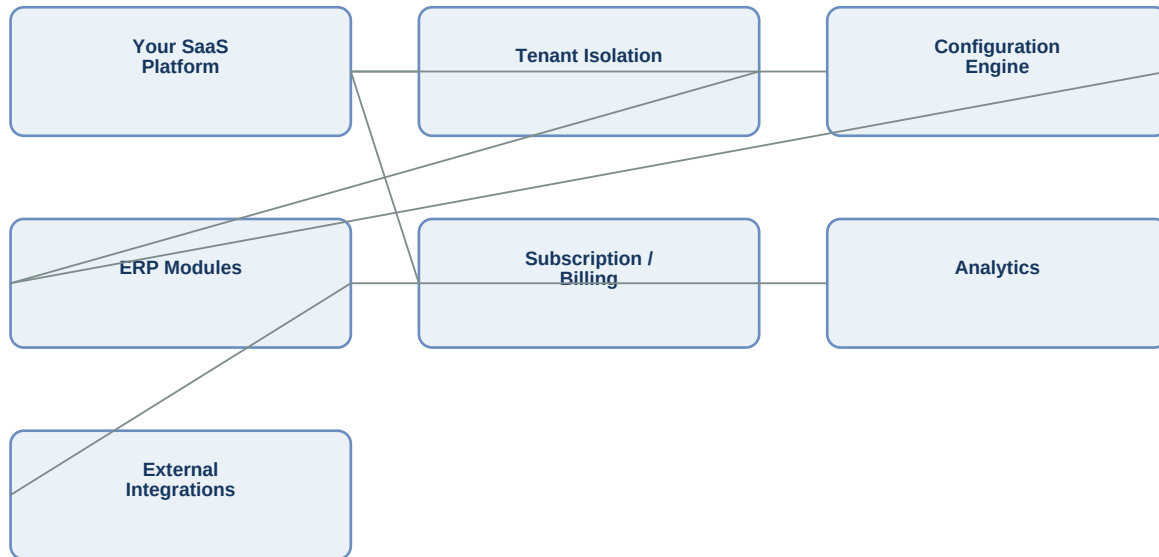


Target: launch with one college, prove the workflows, then onboard hundreds of colleges without creating a separate codebase for each customer.

1. Product Vision

Build a commercial College ERP SaaS rather than a one-off college project. The platform should provide a shared product core, isolated tenant data, configurable workflows, subscription-controlled modules, and a SaaS control plane for your company.

Commercial product architecture



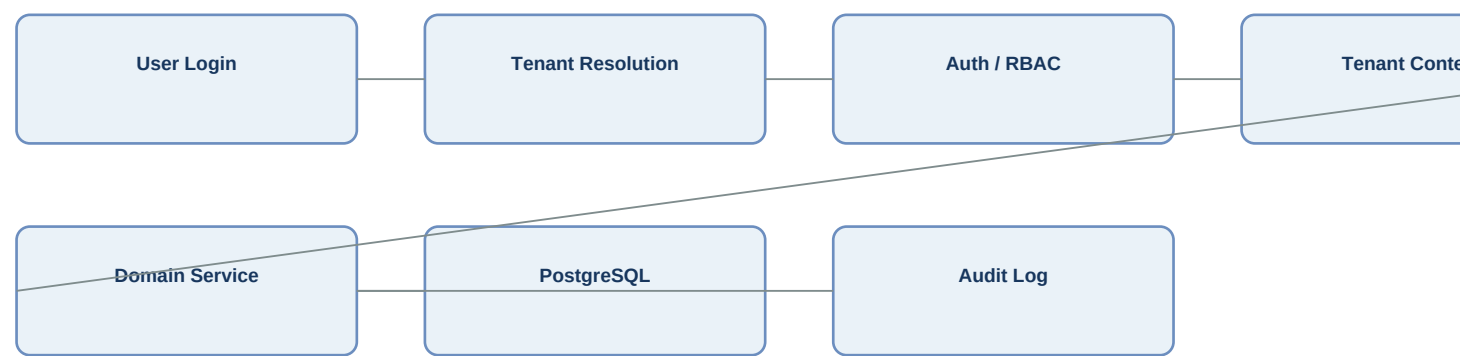
Core commercial principle: customer-specific requirements should be handled by configuration, feature flags and scoped permissions wherever possible—not by branching the codebase.

2. Recommended Language & Technology

Layer	Recommended technology	Reason
Primary language	TypeScript	One language across frontend and backend; strong typing and maintainability for a large ERP.
Frontend	React + TypeScript	Component-based, scalable admin and self-service portals.
Backend	Node.js + NestJS + TypeScript	Modular architecture, validation, dependency injection, REST APIs and background jobs.
Database	PostgreSQL	Strong relational integrity, transactions, constraints and reporting.
ORM	Prisma ORM	Type-safe database access and migrations.
Cache / queue	Redis + BullMQ	Caching, asynchronous jobs, notification queues and retries.
File storage	S3-compatible storage / MinIO	Documents, certificates, photos and attachments.
API	REST + OpenAPI/Swagger	Stable integrations with payment, LMS, accounting and campus systems.
Auth	OIDC/OAuth2 + JWT + RBAC	Secure identity, optional SSO/MFA and role-based access.
Deployment	Docker + Linux + managed cloud services	Repeatable deployments and straightforward scaling.

3. Multi-Tenant Architecture

Tenant-aware request flow



Every tenant-scoped transaction must carry an authenticated tenant context. The backend—not the frontend—must enforce tenant isolation.

Strategy	Use
Shared DB + tenant_id	Recommended starting point. Every tenant-owned table contains tenant_id and every query is tenant-scoped.
Dedicated schema	Optional for larger customers requiring stronger logical separation.
Dedicated DB	Enterprise option for large universities or contractual isolation requirements.

Never rely only on UI filtering. Add service/repository guards, database constraints/policies where appropriate, automated cross-tenant security tests and audit logging.

4. SaaS Control Plane vs College ERP

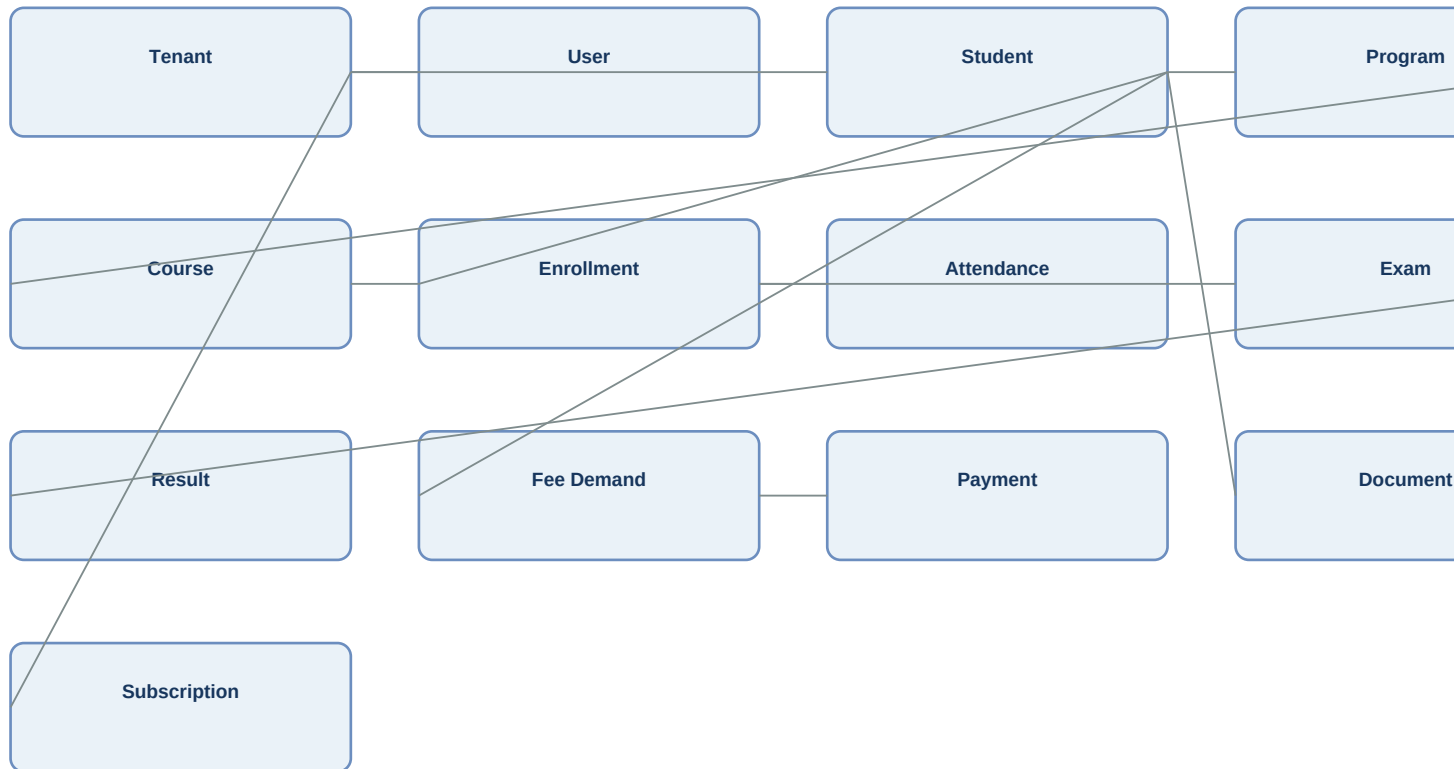
Your SaaS Control Plane	Customer College Tenant
Colleges / tenants	Students
Plans / subscriptions	Faculty / staff
Billing / invoices	Admissions
Feature flags	Academics
Tenant provisioning	Attendance
Usage metering	Fees / payments
System monitoring	Examinations / results
Support / tickets	Library / hostel / transport
Global audit / platform admin	Reports / dashboards

5. Product Module Map

Module	Commercial product scope
Foundation	Tenant, users, roles, permissions, academic year, organization, audit.
Admissions	Lead, application, documents, verification, merit, counselling, admission, enrollment.
Student 360	Profile, guardians, documents, status, academic history, holds, certificates.
Academics	Programs, departments, courses, curriculum, sections, registration, advising.
Timetable	Rooms, slots, faculty availability, conflicts, publishing, substitutions.
Attendance	Period/subject attendance, rules, shortage alerts, optional biometric integration.
Fees	Fee structures, demands, invoices, payments, concessions, scholarships, refunds, reconciliation.
Exams	Exam setup, eligibility, registration, hall ticket, seating, marks, moderation, results.
Certificates	Grade cards, transcripts, provisional certificates, degree workflows.
Library	Catalog, copies, issue/return, reservations, fines.
Hostel / Transport	Allocation, occupancy, routes, vehicles, passes and related fees.
HR / Faculty	Employee master, workload, leave, attendance, payroll integration.
Placements	Companies, drives, eligibility, applications, interviews, offers, outcomes.
Inventory / Assets	Purchasing, vendors, stock, assets, maintenance.
Helpdesk / Feedback	Tickets, escalation, feedback and surveys.
Notifications	Email/SMS/WhatsApp provider integrations, templates, targeted campaigns.
Analytics	Management KPIs, operational reports, exports, scheduled reports.
Integrations	Payment gateway, accounting, LMS, biometric/RFID, identity/document services.

6. Core SaaS Database Design

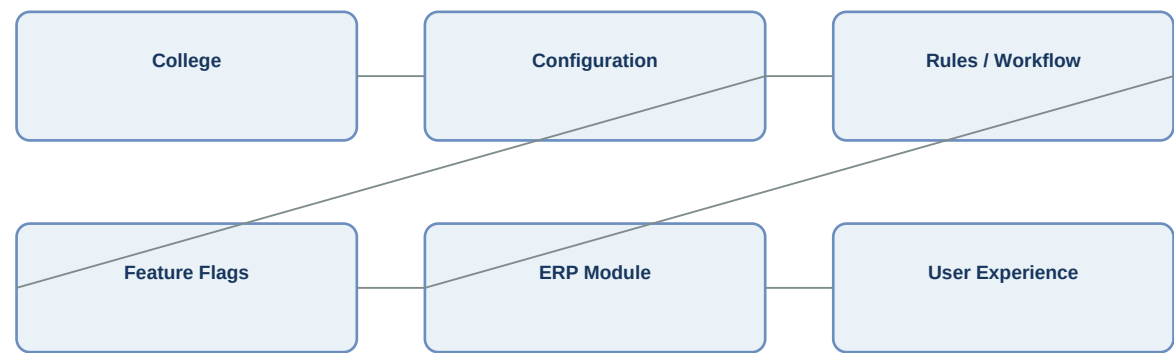
Canonical data model



Domain	Representative tables
SaaS	tenants, plans, subscriptions, subscription_items, feature_flags, usage_events, platform_audit_logs
Identity	users, roles, permissions, user_roles, sessions, login_events
Organization	campuses, departments, programs, academic_years, terms, rooms
Admissions	enquiries, applicants, applications, application_documents, merit_lists, offers
Students	students, guardians, student_documents, student_status_history, holds
Academics	courses, curriculum_versions, curriculum_courses, batches, sections, course_offerings, enrollments
Attendance	attendance_sessions, attendance_records, attendance_rules
Exams	exam_sessions, exam_registrations, eligibility, admit_cards, seating, marks, moderation, results
Finance	fee_heads, fee_structures, fee_demands, invoices, payments, refunds, scholarships, concessions
Services	library_items, library_loans, hostels, hostel_rooms, allocations, transport_routes, vehicles, inventory_items, assets
Communication	notification_templates, notifications, delivery_logs, broadcasts
Audit	audit_logs, approval_actions, workflow_events

7. Configurability Engine

Configuration-driven ERP

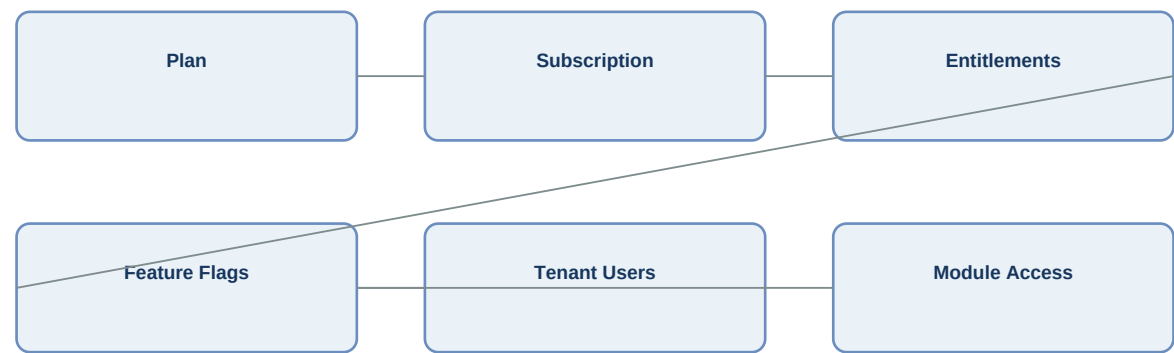


Configuration examples: grading scale, attendance shortage threshold, fee heads, fee schedules, admission form fields, approval chain, certificate templates, academic calendar, notification templates, numbering formats, document checklist and program-specific rules.

Configuration area	Example
Academic	Semester vs trimester, credits, grading scale, passing rules.
Admission	Required documents, application fields, verification stages, approval steps.
Attendance	Minimum percentage, condonation workflow, warning thresholds.
Finance	Fee heads, due dates, late fees, concession rules, refund rules.
Examination	Exam types, mark components, moderation, publication approval.
Communication	Templates, channels, sender identity, audience filters.
Branding	Logo, colors, email templates, certificate layouts, portal branding.

8. Subscription & Feature Management

Commercial access flow

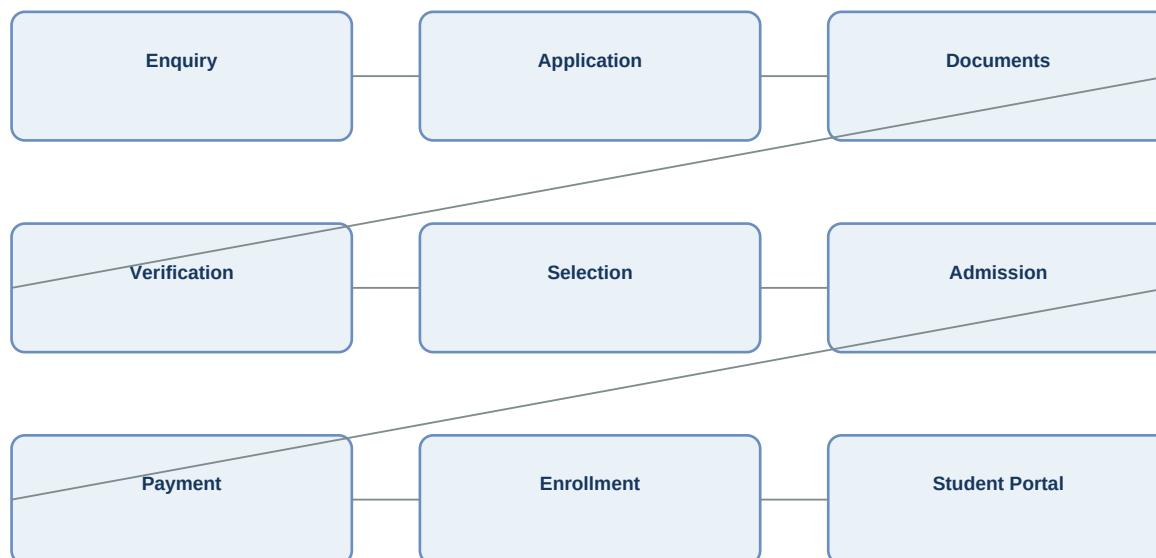


Plan	Possible modules
Starter	Student 360, Academics, Attendance, basic reports.
Professional	Starter + Admissions, Fees, Exams, Notifications.
Enterprise	Professional + Library, Hostel, Transport, HR, Placement, advanced analytics.
University / Custom	Multi-campus, advanced integrations, dedicated infrastructure and custom SLAs.

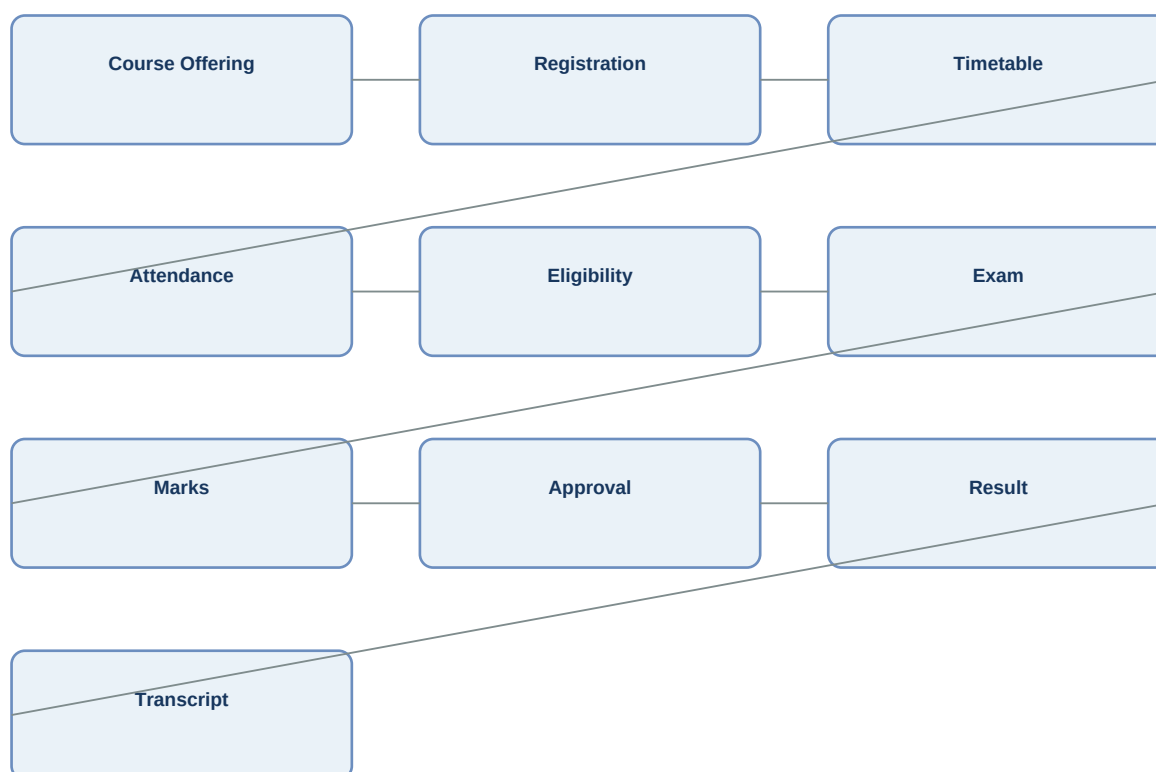
Keep plan names and prices configurable. Your product should support per-student, per-campus, annual-license or negotiated pricing without rewriting module logic.

9. Key Workflows

Admission → student

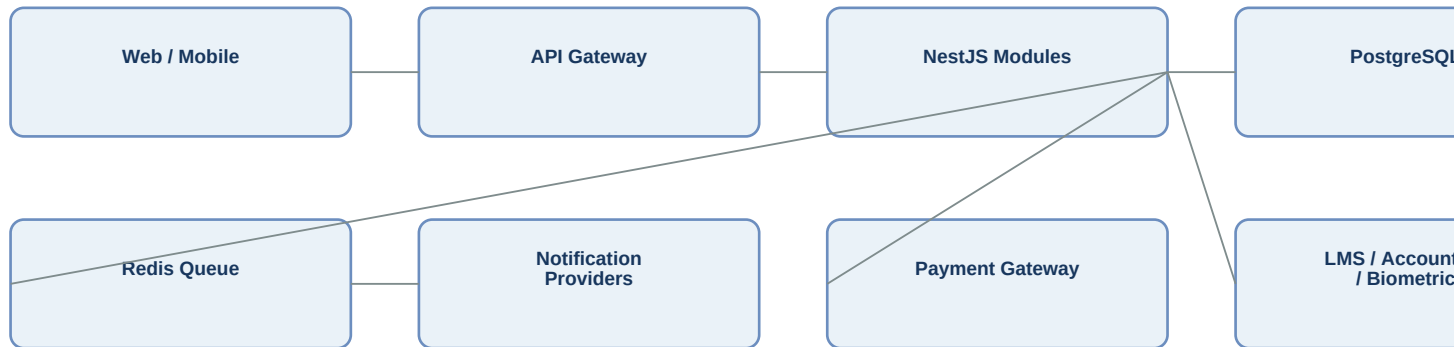


Academic → result



10. API & Integration Architecture

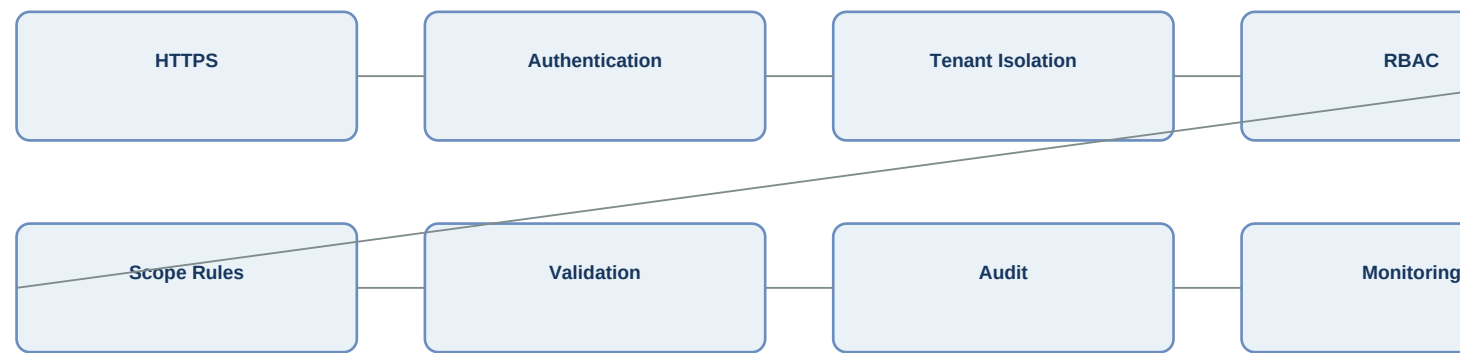
API / event architecture



API group	Examples
Auth	/auth/login, /auth/refresh, /me, /permissions
Tenant	/tenants, /tenant/settings, /tenant/features
Students	/students, /students/:id/documents, /students/:id/timeline
Admissions	/applications, /applications/:id/verify, /applications/:id/approve
Academics	/programs, /courses, /course-offerings, /enrollments
Attendance	/attendance/sessions, /attendance/records
Exams	/exam-sessions, /exam-registrations, /marks, /results/publish
Finance	/fee-demands, /payments/initiate, /payments/webhook, /refunds
SaaS billing	/plans, /subscriptions, /billing/invoices, /usage

11. Security Architecture

Security layers



Control	Requirement
Tenant security	Backend tenant context, isolation tests, no cross-tenant access.
RBAC	Roles + permissions + department/program/campus scope.
Audit	Log approvals, marks changes, refunds, permissions, identity changes and sensitive operations.
Documents	Private storage, signed access URLs, MIME validation, virus scanning and retention controls.
Payments	Verified gateway webhooks, idempotency, reconciliation and immutable transaction references.
Database	Constraints, transactions, backups, restore tests and migration controls.
Operations	Secrets management, dependency updates, monitoring, rate limits and incident response.

12. Development Roadmap

Phase	Goal	Priority
0	Discovery, requirements, domain model, tenant strategy, UX and security design	Must
1	SaaS foundation: tenant, auth, RBAC, subscription, feature flags, audit	Must
2	Organization + master data + Student 360	Must
3	Admissions + onboarding	Must
4	Academics + curriculum + registration + timetable	Must
5	Attendance	Must
6	Fees + payment gateway + reconciliation	Must
7	Exams + results + certificates	Must
8	Library / hostel / transport / inventory / helpdesk	Should
9	HR + placement + analytics	Should
10	Integrations, mobile app, advanced reporting, scaling	Later

13. MVP to Commercial Product

Release	What to ship
MVP — 1 college	Tenant foundation, users/RBAC, student, admissions, academics, attendance, fees, exams/results, basic reports.
v1 — repeatable sales	Self-service tenant onboarding, subscription/feature management, configurable rules, branded portal, payment integration, audit, support.
v2 — multi-college scale	Advanced analytics, integrations, mobile/PWA, bulk imports, automated communications, multi-campus.
v3 — enterprise	Dedicated DB option, SSO, advanced SLA/monitoring, data warehouse, custom workflows and enterprise integrations.

14. Folder Structure

apps/

web/ — React + TypeScript

api/ — NestJS + TypeScript

worker/ — background jobs

packages/

database/ — Prisma schema and migrations

auth/ — shared auth contracts

ui/ — reusable UI components

config/ — configuration schemas

types/ — shared TypeScript types

api/src/modules/

saas/ auth/ tenants/ users/ admissions/ students/ academics/ timetable/ attendance/

fees/ payments/ exams/ results/ certificates/ library/ hostel/ transport/

hr/ placements/ inventory/ notifications/ reports/ integrations/ audit/

Keep each domain modular. A module should own its business rules, service layer, API handlers, database access and tests. Avoid a single giant controller/service containing every college workflow.

15. Scaling from 1 to Hundreds of Colleges

Scale path



Scale	Architecture focus
1 college	Shared PostgreSQL, modular monolith, one production environment.
10–50	Connection pooling, Redis, worker queues, object storage, centralized monitoring, automated onboarding.
50–100+	Read replicas/analytics separation, stronger caching, queue partitioning, rate limits, autoscaling.
Enterprise	Optional dedicated database/schema, SSO, stronger isolation, custom integrations and SLA controls.

16. Testing & Quality Gates

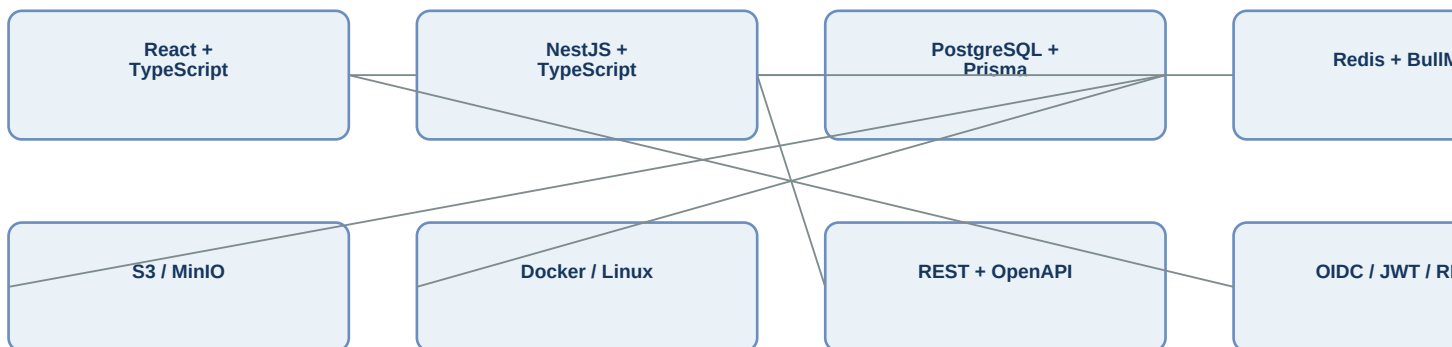
Test level	Examples
Unit	Fee calculation, grade calculation, attendance rules, permissions, status transitions.
Integration	Payment webhook, notification provider, accounting/LMS/biometric connectors.
Tenant isolation	Automated tests attempting cross-tenant reads/writes for every sensitive API.
Workflow/UAT	Admission, fee payment, exam publication, certificate, refund and approval workflows.
Security	IDOR, authorization bypass, file access, injection, session handling, rate limiting.
Performance	Admission spikes, bulk attendance, result publication, report generation.
Recovery	Database restore, queue recovery, document recovery and rollback drills.

17. Commercial Go-Live Checklist

- Tenant onboarding works without developer database changes.
- Every tenant-scoped table/query is protected by tenant isolation.
- Plans and feature entitlements can be changed without code deployment.
- College branding/configuration can be changed from the admin portal.
- Payment success/failure/pending/refund flows are reconciled.
- Critical exam and result actions are audited and approval-controlled.
- Backups and restore tests are completed.
- Monitoring, error tracking and alerting are operational.
- Migration tools support repeatable import from Excel/legacy ERP.
- Support workflow and SLA ownership are defined.
- Pilot college signs off before wider sales rollout.

18. Final Recommended Stack

Production stack



Final answer: use **TypeScript** as the main development language, **React** for the frontend, **NestJS/Node.js** for the backend, and **PostgreSQL** as the primary database. Add Redis for cache/queues and S3-compatible storage for files. Build multi-tenancy, configuration, feature flags and subscription entitlements from the beginning.

Do not start with microservices. Start with a modular monolith and clean domain boundaries. Extract services only when real scale or team boundaries require it.